

Chapitre 4: Modèles de régression pour données binaires

Les modèles vus dans les deux chapitres précédents ne permettent de considérer que les cas où la variable réponse Y_i est continue, puisqu'on supposait que Y_i suivait une loi normale. Cependant, dans plusieurs situations, on a une variable réponse discrète qui ne peut prendre qu'un nombre fini (ou dénombrable) de valeurs possibles. Dans ce chapitre, on s'intéresse au cas où Y_i suit une loi de Bernoulli ou une loi binomiale. Voici quatre exemples:

- Ex 1:** Considérons le jeu de données `quilpie` de la library `GLMsData` du logiciel R. Ce jeu de données contient des observations de 1921 à 1988 de la quantité de pluie mesurée durant le mois de juillet à Quilpie en Australie. Le jeu de données contient des variables explicatives qui concernent la pression atmosphérique. La variable réponse ne peut prendre que 2 valeurs: 1 si la quantité de pluie mesurée est supérieure à 10 millimètres et 0 sinon.
- Ex 2:** Considérons le jeu de données `nminer` de la library `GLMsData` du logiciel R. Ce jeu de données contient 31 observations de 9 variables. La variable réponse `Mminers` ne peut prendre que 2 valeurs: 1 si un mineur bruyant est détecté dans la parcelle de terre et 0 sinon.
- Ex 3:** Considérons le jeu de données `deposit` de la library `GLMsData` du logiciel R. Ce jeu de données contient 18 observations avec deux variables explicatives : type (A , B ou C) et dose d'insecticide. La variable réponse Y_i est le nombre d'insectes tués parmi les m_i insectes vivants au début de l'expérience.
- Ex 4:** Considérons le jeu de données `turbines` de la library `GLMsData` du logiciel R. Ce jeu de données contient 11 observations avec une variable explicative : nombre d'heures de fonctionnement. La variable réponse Y_i est le nombre machines qui présentent des fissures parmi les m_i machines neuves au début de l'expérience.

1 Données et modèle

1.1 Données individuelles

Dans les deux premiers exemples, la variable réponse Y_i ne peut prendre que deux valeurs possibles: 0 et 1. Les données sont donc sous la forme

$\{(Y_i, X_i), i = 1, \dots, n\}$; où n est nombre d'observations, $Y_i \in \{0, 1\}$ est la variable réponse et $X_i = (X_{i1}, \dots, X_{ip})^\top$ est l'ensemble des variables explicatives pour l'observation i . On dit que Y_i suit une loi de Bernoulli avec un paramètre $\pi_i \in [0, 1]$. On pose alors $P(Y_i = 1) = \pi_i$ et $P(Y_i = 0) = 1 - P(Y_i = 1) = 1 - \pi_i$ et on a

$$\begin{aligned} P(Y_i = y_i) &= \pi_i^{y_i} (1 - \pi_i)^{1-y_i}, \quad y_i \in \{0, 1\} \\ E[Y_i] &= \mu_i = \pi_i \\ Var[Y_i] &= \sigma_i^2 = \pi_i(1 - \pi_i). \end{aligned}$$

D'autres part, on pose $\eta_i = \beta_0 + \beta_1 X_{i1} + \dots + \beta_p X_{ip}, i = 1, \dots, n$. Ici p est le nombre de variables explicatives considérées et $\beta = (\beta_0, \beta_1, \dots, \beta_p)^\top$ est un vecteur de $p + 1$ coefficient de régression. à estimer à parti des données. Finalement, on a $\eta_i = g(\pi_i)$; où $g(\cdot)$ est une fonction de lien, supposée connue. La fonction de lien la plus populaire est

$$g(u) = \text{logit}(u) = \log \left(\frac{u}{1-u} \right).$$

Avec cette fonction de lien, on a

$$\pi_i = P(Y_i = 1) = \frac{e^{\beta_0 + \beta_1 X_{i1} + \dots + \beta_p X_{ip}}}{1 + e^{\beta_0 + \beta_1 X_{i1} + \dots + \beta_p X_{ip}}}.$$

Et donc, si $\beta_j > 0$, une hausse de X_{ij} avec toutes les autres variables explicatives maintenues constantes augmentera la probabilité $P(Y_i = 1)$. Si $\beta_j < 0$, une hausse de X_{ij} avec toutes les autres variables explicatives maintenues constantes diminuera la probabilité $P(Y_i = 1)$. Finalement, si $\beta_j = 0$, alors X_{ij} n'a pas d'effets sur la probabilité $P(Y_i = 1)$.

1.2 Données groupées

Dans les deux derniers exemples, la variable réponse Y_i peut prendre n'importe quelle valeur dans l'ensemble $\{0, 1, \dots, m_i\}$, où m_i est un entier connu à l'avance et qui varie d'une observation à une autre. 'A toute fin pratique, m_i peut être considéré comme un nombre total d'expériences indépendantes ayant deux résultats possibles (succès ou échec) et Y_i est interprété comme le nombre de succès parmi les m_i expériences. Les données sont donc sous la forme $\{(Y_i, m_i, X_i), i = 1, \dots, n\}$. On dit que Y_i suit une loi Binomiale de

paramètres m_i et $\pi_i \in [0, 1]$ et on écrit $Y_i \sim \text{Bin}(m_i, \pi_i)$. Selon cette loi, on a:

$$\begin{aligned} P(Y_i = y_i) &= \binom{m_i}{y_i} \pi_i^{y_i} (1 - \pi_i)^{m_i - y_i}, \quad y_i \in \{0, 1, \dots, m_i\} \\ E[Y_i] &= \mu_i = m_i \pi_i \\ \text{Var}[Y_i] &= \sigma_i^2 = m_i \pi_i (1 - \pi_i) = \frac{\mu_i(m_i - \mu_i)}{m_i}. \end{aligned}$$

Pour relier la variable réponse aux variables explicative, on pose $\eta_i = g(\pi_i)$, $i = 1, \dots, n$. Ce choix est motivé par le fait que π_i doit appartenir à l'intervalle $[0, 1]$.

Si on a $m_1 = m_2 = \dots = m_n = 1$, on retrouve la loi de Bernoulli des deux premiers exemples. La loi de Bernoulli est un cas particulier de la loi binomiale. Dans la suite de ce chapitre, on se contentera de présenter les développements pour la loi binomiale.

1.3 Fonctions de lien

La fonction de lien g spécifiée pour des données binaires doit s'assurer que $\pi_i = g^{-1}(\eta_i)$ appartient à l'intervalle $[0, 1]$, puisque π_i est une probabilité. Trois fonctions de lien sont communément utilisées pour les données binaires:

Logit: Cette fonction est définie par

$$g(u) = \text{logit}(u) = \log\left(\frac{u}{1-u}\right).$$

Ainsi, on a $\pi_i = g^{-1}(\eta_i) = e^{\eta_i} / (1 + e^{\eta_i})$. Avec logiciel R, on spécifie `link="logit"` à l'intérieur de la fonction `glm`.

Probit: Cette fonction est définie par $g(u) = \Phi^{-1}(u)$, où Φ est la fonction de répartition de la loi normale standard $\mathcal{N}(0, 1)$. Ainsi, on a $\pi_i = g^{-1}(\eta_i) = \Phi(\eta_i)$. Avec logiciel R, on spécifie `link="probit"` à l'intérieur de la fonction `glm`.

c-log-log: Cette fonction est définie par $g(u) = \log\{-\log(1-u)\}$. Ainsi, on a $\pi_i = g^{-1}(\eta_i) = 1 - e^{-e^{\eta_i}}$. Avec logiciel R, on spécifie `link="cloglog"` à l'intérieur de la fonction `glm`.

On remarque que dans les trois cas présentés ci-haut, $g^{-1}(\cdot)$ est une fonction de répartition.

2 Inférence statistique

2.1 Estimation des paramètres β

Le vecteur des paramètres du modèle est $\beta = (\beta_0, \beta_1, \dots, \beta_p)^\top$. La fonction de log-vraisemblance est

$$\begin{aligned} \sum_{i=1}^n \log [P(Y_i = y_i)] &= \sum_{i=1}^n \log \left[\binom{m_i}{y_i} \pi_i^{y_i} (1 - \pi_i)^{m_i - y_i} \right] \\ &= \sum_{i=1}^n \left\{ \log \left[\binom{m_i}{y_i} \right] + y_i \log(\pi_i) + (m_i - y_i) \log(1 - \pi_i) \right\}. \end{aligned}$$

Cette fonction dépend du vecteur de paramètres β à travers π_i , puisqu'on a

$$\pi_i = g^{-1}(\eta_i) = g^{-1}(\beta_0 + \beta_1 X_{i1} + \dots + \beta_p X_{ip}).$$

En remplaçant π_i par μ_i/m_i dans l'expression précédente, on montre que cette fonction log-vraisemblance est égale à

$$l(\mu; y) = \sum_{i=1}^n \{y_i \log(\mu_i) + (m_i - y_i) \log(m_i - \mu_i)\} + K$$

où

$$K = \sum_{i=1}^n \left\{ \log \left[\binom{m_i}{y_i} \right] - m_i \log(m_i) \right\}$$

est une constante qui ne dépend pas de β . L'estimateur de maximum de vraisemblance de β , disons $\hat{\beta}$, est la valeur de β qui maximise $l(\beta; y, \mu)$. On calcule par la suite la matrice d'information de Fisher I . L'élément à la position (j, k) de la matrice I est égal à

$$-\frac{\partial^2 l(\beta; y, \mu)}{\partial \beta_j \partial \beta_k}$$

évaluée à $\beta = \hat{\beta}$. La matrice de variance-covariance de $\hat{\beta}$ est alors estimée par $v(\hat{\beta}) = I^{-1}$. Par la théorie générale de l'estimation par maximum de vraisemblance, la distribution asymptotique de $\hat{\beta}$ est approximée par la loi normale multivariée $\mathcal{N}\{\beta, v(\hat{\beta})\}$.

On montre que $v(\hat{\beta}) = (X^\top W X)^{-1}$ où X est une matrice $n \times (p+1)$ contenant les variables explicatives, incluant l'intercepte et W est une matrice diagonale de taille n avec les entrées $w_i = \hat{\mu}_i(m_i - \hat{\mu}_i)/m_i, i = 1, \dots, n$ sur la diagonale. Ici, on a

$$\hat{\mu}_i = m_i \hat{\pi}_i = m_i g^{-1}(\hat{\eta}_i) = m_i g^{-1}(\hat{\beta}_0 + \hat{\beta}_1 X_{i1} + \dots + \hat{\beta}_p X_{ip})$$

2.2 Tests d'hypothèses et intervalles de confiance

Les intervalles de confiance et les tests d'hypothèses concernant les β_j sont obtenus par la méthode de Wald.

Ainsi, un intervalle de confiance à $100 \times (1 - \alpha)\%$ pour β_j est donné par

$$[\hat{\beta}_j - z_{\alpha/2} \sqrt{v(\hat{\beta}_j)}; \hat{\beta}_j + z_{\alpha/2} \sqrt{v(\hat{\beta}_j)}],$$

où z_γ est le quantile de la loi normale standard défini par $P(Z > z_\gamma) = \gamma$, $\gamma \in (0, 1)$, $Z \sim \mathcal{N}(0, 1)$ et $v(\hat{\beta}_j)$ est l'élément $(j+1, j+1)$ de la matrice $v(\hat{\beta})$, $j = 0, 1, \dots, p$.

Considérons maintenant le test d'hypothèse $H_0 : \beta_j = 0$ versus $H_1 : \beta_j \neq 0$.

On rejette H_0 au seuil α si $|z_{obs}| > z_{\alpha/2}$, avec $z_{obs} = \hat{\beta}_j / \sqrt{v(\hat{\beta}_j)}$. Le seuil observé (p -valeur ou p -value) est $\alpha^* = P(|Z| > |z_{obs}|) = 2P(Z > |z_{obs}|)$, $Z \sim \mathcal{N}(0, 1)$.

Intéressons nous maintenant à π^* lorsque $X = X^* = (1, X_1^*, \dots, X_p^*)^\top$. On estime π par $\hat{\pi}^* = g^{-1}(\hat{\beta}_0 + \hat{\beta}_1 X_1^* + \dots + \hat{\beta}_p X_p^*) = g^{-1}(X^* \hat{\beta})$.

Un intervalle de confiance à $100 \times (1 - \alpha)\%$ pour $X^* \beta$ est donné par

$$\left[X^* \hat{\beta} - z_{\alpha/2} \sqrt{v(X^* \hat{\beta})}, X^* \hat{\beta} + z_{\alpha/2} \sqrt{v(X^* \hat{\beta})} \right],$$

avec $v(X^* \hat{\beta}) = X^* v(\hat{\beta}) X^{*\top}$. On en déduit un intervalle de confiance à $100 \times (1 - \alpha)\%$ pour π^* :

$$\left[g^{-1} \left\{ X^* \hat{\beta} - z_{\alpha/2} \sqrt{v(X^* \hat{\beta})} \right\}, g^{-1} \left\{ X^* \hat{\beta} + z_{\alpha/2} \sqrt{v(X^* \hat{\beta})} \right\} \right].$$

Alternativement, on peut approximer $\text{Var}(\hat{\pi}^*)$ par la règle du delta. On obtient $v(\hat{\pi}^*) = h(X^* \hat{\beta}) v(X^* \hat{\beta})$, où $h(u) = \{\partial g^{-1}(u) / \partial u\}^2$.

Un intervalle de confiance approximatif à $100 \times (1 - \alpha)\%$ pour π^* est donné par

$$\left[\hat{\pi}^* - 1.96 \sqrt{v(\hat{\pi}^*)}, \hat{\pi}^* + 1.96 \sqrt{v(\hat{\pi}^*)} \right].$$

2.3 Exemple en R

Considérons le jeu de données turbines de la library `GLMsData` du logiciel R.

```
> if (!require("GLMsData")) install.packages("GLMsData")
> library("GLMsData")#### Installer la librairie GLMsData
>
```

```

> data(turbines)#### Rendre le jeu de donnees turbines disponible
> attach(turbines)
The following objects are masked from turbines (pos = 4):

    Fissures, Hours, Turbines

>
> #####
> ## Ajustement du modele avec trois fonctions de lien differents ##
> #####
>
> modele.logit <- glm(cbind(Fissures,Turbines-Fissures)~Hours,
                      data=turbines,family=binomial(link="logit"))
> modele.probit <- glm(cbind(Fissures,Turbines-Fissures)~Hours,
                      data=turbines,family=binomial(link="probit"))
> modele.cloglog <- glm(cbind(Fissures,Turbines-Fissures)~Hours,
                      data=turbines,family=binomial(link="cloglog"))
>
> resultats <- round(rbind(summary(modele.logit)$coef,
                           summary(modele.probit)$coef,
                           summary(modele.cloglog)$coef),6)
>
> resultats <- resultats[c(1,3,5,2,4,6),]
>
> row.names(resultats) <- c("beta0 (logit)", "beta0 (probit)","beta0 (cloglog)",
                           "beta1 (logit)", "beta1 (probit)","beta1 (cloglog)")
>
> resultats

```

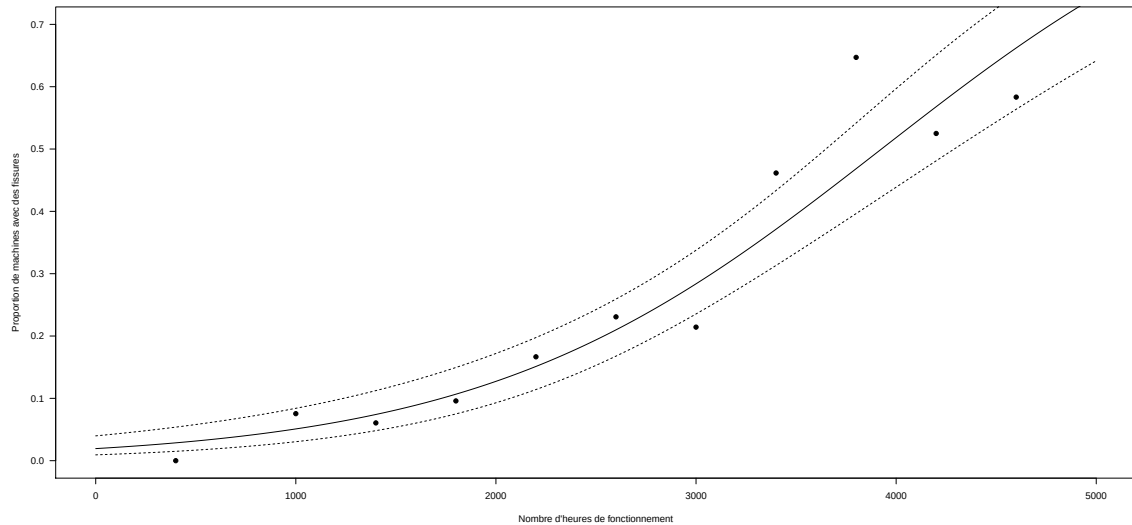
	Estimate	Std. Error	z value	Pr(> z)
beta0 (logit)	-3.923597	0.377959	-10.381013	0
beta0 (probit)	-2.275807	0.197419	-11.527823	0
beta0 (cloglog)	-3.603280	0.324739	-11.095923	0
beta1 (logit)	0.000999	0.000114	8.753682	0
beta1 (probit)	0.000578	0.000063	9.238777	0
beta1 (cloglog)	0.000810	0.000090	8.992278	0

Les trois fonctions de lien donnent des résultats très similaires, ce qui est très fréquent.

Traçons maintenant $\hat{\pi}^*$ avec les intervalles de confiance pour différentes valeurs du nombre d'heures de fonctionnement. On utilise le lien logit et

on calcule les intervalles de confiance de deux manières différentes, comme détaillé ci-haut. Voici le code R pour la première méthode:

```
#####  
## On trace la courbe de pi predict en fonction du nombre d'heures      ##  
## de fonctionnement avec les intervalles de confiance pour le lien logit ##  
#####  
  
## 1ere facon: intervalles de confiance exacts  
  
Heures <- 0:5000  
nouvelles.donnees <- predict(modele.logit,  
                             newdata=data.frame(Hours=Heures),se.fit=TRUE)  
nu.chapeau <- nouvelles.donnees$fit  
se.nu.chapeau <- nouvelles.donnees$se.fit  
pi.chapeau <- exp(nu.chapeau)/(1+exp(nu.chapeau))  
A <- exp(nu.chapeau-1.96*se.nu.chapeau)  
B <- exp(nu.chapeau+1.96*se.nu.chapeau)  
intervalle.confiance.1.pi.chapeau <- A/(A+1)  
intervalle.confiance.2.pi.chapeau <- B/(B+1)  
  
plot(Hours,Fissures/Turbines,xlab="Nombre d'heures de fonctionnement",pch=19,las=1,  
      xlim=c(0,5000),ylim=c(0,0.7),ylab="Proportion de machines avec des fissures")  
lines(Heures,pi.chapeau)  
lines(Heures,intervalle.confiance.1.pi.chapeau,lty=2)  
lines(Heures,intervalle.confiance.2.pi.chapeau,lty=2)  
  
et on obtient
```

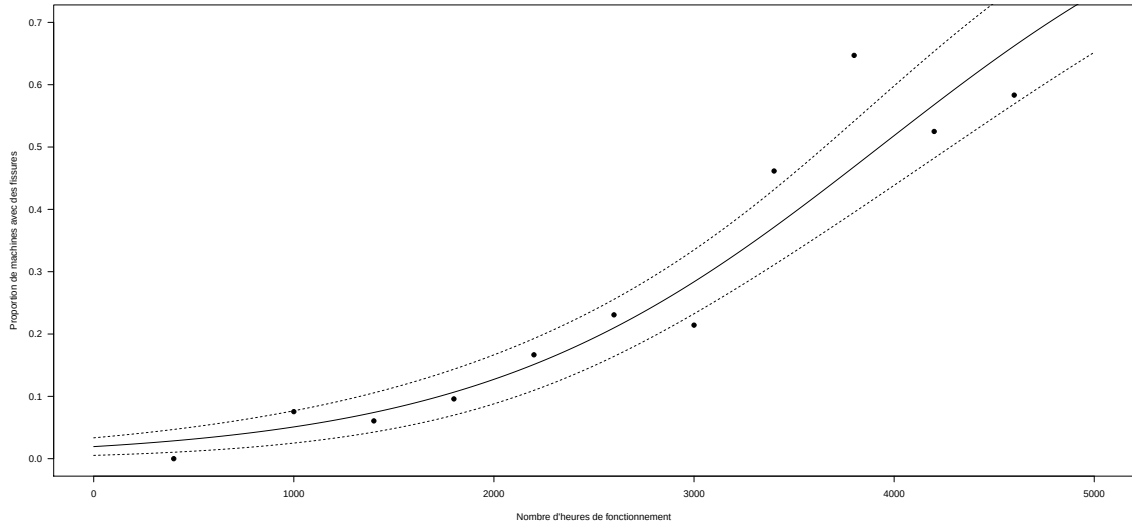


Et voici le code R pour la deuxième méthode:

```
## 2eme facon: intervalles de confiance approximatifs
```

```
Heures <- 0:5000
nouvelles.donnees <- predict(modele.logit,newdata=data.frame(Hours=Heures),
                             se.fit=TRUE,type="response")
pi.chapeau <- nouvelles.donnees$fit
se.pi.chapeau <- nouvelles.donnees$se.fit
plot(Heures,Fissures/Turbines,xlab="Nombre d'heures de fonctionnement",pch=19,las=1,
     xlim=c(0,5000),ylim=c(0,0.7),ylab="Proportion de machines avec des fissures")
lines(Heures,pi.chapeau)
lines(Heures,pi.chapeau-1.96*se.pi.chapeau,lty=2)
lines(Heures,pi.chapeau+1.96*se.pi.chapeau,lty=2)
```

et on obtient



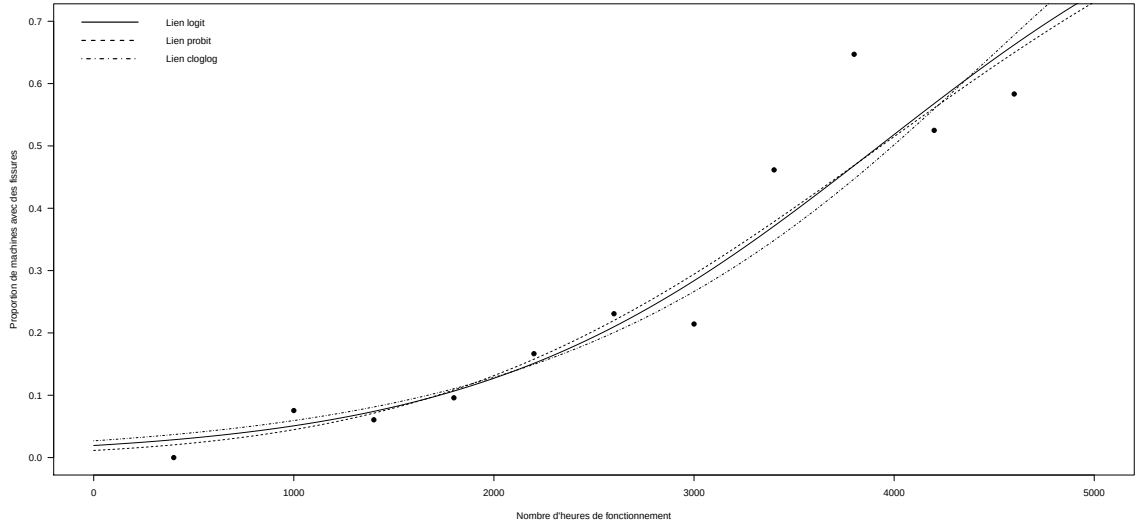
Les deux graphiques sont très similaires.

Traçons maintenant $\hat{\pi}^*$ pour différentes valeurs du nombre d'heures de fonctionnement en considérant les 3 fonctions de lien.

```
#####
## On trace la courbe de pi predict en fonction du nombre d'heure ##
## de fonctionnement pour les liens logit, probit et cloglog      ##
#####
```

```
Heures <- 0:5000
nouvelles.donnees.logit <- predict(modele.logit,newdata=data.frame(Heures=Heures),
                                   se.fit=TRUE,type="response")
nouvelles.donnees.probit <- predict(modele.probit,newdata=data.frame(Heures=Heures),
                                    se.fit=TRUE,type="response")
nouvelles.donnees.cloglog <- predict(modele.cloglog,newdata=data.frame(Heures=Heures),
                                     se.fit=TRUE,type="response")
plot(Heures,Fissures/Turbines,xlab="Nombre d'heures de fonctionnement",pch=19,las=1,
     xlim=c(0,5000),ylim=c(0,0.7),ylab="Proportion de machines avec des fissures")
lines(Heures,nouvelles.donnees.logit$fit,lty=1)
lines(Heures,nouvelles.donnees.probit$fit,lty=2)
lines(Heures,nouvelles.donnees.cloglog$fit,lty=4)
legend("topleft",lwd=2,lty=c(1,2,4),
      legend=c("Lien logit","Lien probit", "Lien cloglog"),bty="n")
```

et on obtient



Les courbes obtenues avec les trois fonctions de lien sont très similaires. Ce qui est très fréquent.

3 Sélection de modèles

3.1 Déviance

Considérons maintenant un modèle, dit saturé, avec n paramètres $\{\tilde{\mu}_1, \dots, \tilde{\mu}_n\}$ et sans variables explicatives. Sous ce modèle, on a

$$P(Y_i = y_i) = \binom{m_i}{y_i} \frac{\tilde{\mu}_i^{y_i} (m_i - \tilde{\mu}_i)^{m_i - y_i}}{m_i^{m_i}}$$

et on montre que les estimateurs de maximum de vraisemblance de $\{\tilde{\mu}_1, \dots, \tilde{\mu}_n\}$ sont $\{y_1, \dots, y_n\}$. La statistique de test de rapport de vraisemblance entre ce modèle saturé et le modèle complet considéré dans la section précédente

est

$$\begin{aligned}
D &= 2 \{l(y, y) - l(\hat{\mu}, y)\} \\
&= 2 \sum_{i=1}^n \left\{ y_i \log \left(\frac{y_i}{\hat{\mu}_i} \right) + (m_i - y_i) \log \left(\frac{m_i - y_i}{m_i - \hat{\mu}_i} \right) \right\} \\
&= 2 \sum_{i=1}^n d_i.
\end{aligned}$$

La statistique D est appelée la déviance du modèle alors que la statistique d_i est la déviance individuelle de l'observation i . La statistique D joue le même rôle que la somme des carrés des résidus pour la régression linéaire. Elle est utilisée pour évaluer l'ajustement et l'adéquation du modèle aux données et pour comparer des modèles.

3.2 Comparaison de modèles emboîtés

Deux modèles sont emboîtés si l'un des deux est un cas particulier de l'autre. Pour comparer deux modèles emboîtés, on commence par ajuster chacun des modèles aux données. On calcule par la suite la déviance de chaque modèle, disons D_1 et D_2 . La statistique de test pour comparer les deux modèles est $\chi^2 = D_2 - D_1$. Sous H_0 : le modèle réduit est plus adéquat aux données, on a χ^2 suit une loi de Khi-deux à d degrés de liberté, où d est la différence entre le nombre de paramètres du modèle complet et celui du modèle réduit. Revenons sur le jeu de données turbines de la library `GLMsData` du logiciel R. On voudrait savoir si c'est pertinent d'ajouter des termes quadratique et cubique avec la fonction de lien logit. Ainsi, avec le modèle complet on a :

$$\pi_i = \frac{e^{\beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \beta_3 x_i^3}}{1 + e^{\beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \beta_3 x_i^3}}$$

alors qu'avec le modèle réduit, on a

$$\pi_i = \frac{e^{\beta_0 + \beta_1 x_i}}{1 + e^{\beta_0 + \beta_1 x_i}}.$$

Comparer ces deux modèles revient à tester l'hypothèse nulle $H_0 : \beta_2 = \beta_3 = 0$. On calcule alors la déviance de chacun de ces modèles. La statistique du test est la différence entre les deux déviances.

```
> data(turbines)#### Rendre le jeu de donnees turbines disponible
> attach(turbines)
```

The following objects are masked from turbines (pos = 3):

Fissures, Hours, Turbines

The following objects are masked from turbines (pos = 4):

Fissures, Hours, Turbines

```
>
> #####
> ## Comparaison de modeles emboites ##
> #####
>
> ## Modele complet avec des termes quadratique
> ## et cubique de la variable explicative Heure
>
> modele.complet <- glm(cbind(Fissures,Turbines-Fissures)~Hours+Hours^2)+
+                       I(Hours^3),data=turbines,family=binomial(link="logit"))
>
> modele.reduit <- glm(cbind(Fissures,Turbines-Fissures)~Hours,data=turbines,
+                       family=binomial(link="logit"))
>
> stat.test <- deviance(modele.reduit)-deviance(modele.complet)
>
> p.valeur <- 1-pchisq(stat.test,df=2)
>
> p.valeur
[1] 0.4767217
```

Le seuil observé du test est 0.47. On ne rejette pas l'hypothèse nulle. On conclut que le modèle réduit est plus adéquat aux données.

3.3 Comparaison de modèles non-emboîtés

Comme pour les modèles linéaires, on compare les modèles non-emboîtés en utilisant les critères *AIC* et *BIC*. Ces critères sont définis comme suit:

$$\begin{aligned} AIC &= -2l(\hat{\mu}; y) + 2(p + 1) \\ BIC &= -2l(\hat{\mu}; y) + \log(n)(p + 1) \end{aligned}$$

Le modèle ayant le plus petit *AIC* (ou *BIC*) est préférable. Considérons le jeu de données `deposit` de la library `GLMsData` du logiciel R. Ce jeu de données comporte deux variables explicatives: `type` et `dose` de l'insecticide utilisé et la variable réponse est le nombre d'insectes tués parmi ceux qui ont commencé l'expérience. On considère deux modèles. Le premier avec les deux variables explicatives et leur interaction. Le deuxième modèle considère la variable `dose` de l'insecticide avec des termes quadratique et cubique. Ces deux modèles ne sont pas emboîtés. On utilise les critères *AIC* et *BIC* pour déterminer celui qui est plus adéquat aux données.

```
> if (!require("GLMsData")) install.packages("GLMsData")
> library("GLMsData")#### Installer la librairie GLMsData
>
> data(deposit)#### Rendre le jeu de donnees deposit disponible
> n <- dim(deposit)[1]
>
>
> modele1 <- glm(cbind(Killed,Number-Killed)~factor(Insecticide)*Deposit,
+               data=deposit,family=binomial)
>
> modele2 <- glm(cbind(Killed,Number-Killed)~Deposit+I(Deposit^2)+I(Deposit^3),
+               data=deposit,family=binomial)
>
> AIC1 <- extractAIC(modele1)[2]; AIC1
[1] 116.9708
> AIC2 <- extractAIC(modele2)[2]; AIC2
[1] 278.0755
>
> BIC1 <- extractAIC(modele1,k=log(n))[2]; BIC1
[1] 122.313
> BIC2 <- extractAIC(modele2,k=log(n))[2]; BIC2
[1] 281.637
```

Selon les critères *AIC* et *BIC*, le premier modèle est plus adéquat aux données.

3.4 Sélection automatique de modèle

Les méthodes pas à pas *forward*, *backward* et *stepwise* fonctionnent exactement comme pour les modèles linéaires. On considère le jeu de données

deposit de la library GLMsData du logiciel R. On utilise ces procédures pour sélectionner le meilleur modèle. Le modèle le plus complet comporte les deux variables explicatives: type et dose de l'insecticide, ainsi que leur interaction et les termes quadratique et cubique de dose.

```
> max.modele <- glm(cbind(Killed,Number-Killed)~factor(Insecticide)*Deposit+
                    I(Deposit^2)+I(Deposit^3),data=deposit,family=binomial)
> min.modele <- glm(cbind(Killed,Number-Killed)~1,data=deposit,family=binomial)
>
> step(min.modele,scope=list(lower=min.modele, upper=max.modele),
      direction="forward")
Start:  AIC=477.84
cbind(Killed, Number - Killed) ~ 1
```

	Df	Deviance	AIC
+ Deposit	1	231.56	297.76
+ I(Deposit^2)	1	258.25	324.44
+ factor(Insecticide)	2	259.41	327.61
+ I(Deposit^3)	1	285.10	351.30
<none>		413.64	477.84

```
Step:  AIC=297.76
cbind(Killed, Number - Killed) ~ Deposit
```

	Df	Deviance	AIC
+ factor(Insecticide)	2	48.026	118.22
+ I(Deposit^3)	1	207.878	276.08
+ I(Deposit^2)	1	208.131	276.33
<none>		231.559	297.76

```
Step:  AIC=118.22
cbind(Killed, Number - Killed) ~ Deposit + factor(Insecticide)
```

	Df	Deviance	AIC
+ I(Deposit^2)	1	14.499	86.696
+ I(Deposit^3)	1	15.160	87.357
+ factor(Insecticide):Deposit	2	42.773	116.971
<none>		48.026	118.223

```
Step:  AIC=86.7
```

```
cbind(Killed, Number - Killed) ~ Deposit + factor(Insecticide) +
  I(Deposit^2)
```

	Df	Deviance	AIC
<none>		14.499	86.696
+ I(Deposit^3)	1	14.406	88.604
+ factor(Insecticide):Deposit	2	14.358	90.556

```
Call: glm(formula = cbind(Killed, Number - Killed) ~ Deposit +
  factor(Insecticide) +
  I(Deposit^2), family = binomial, data = deposit)
```

Coefficients:

(Intercept)	Deposit	factor(Insecticide)B
-6.3808	2.1511	0.3061
factor(Insecticide)C	I(Deposit^2)	
2.9264	-0.1532	

Degrees of Freedom: 17 Total (i.e. Null); 13 Residual

Null Deviance: 413.6

Residual Deviance: 14.5 AIC: 86.7

```
> step(max.modele,scope=list(lower=min.modele, upper=max.modele),
  direction="backward")
```

Start: AIC=92.5

```
cbind(Killed, Number - Killed) ~ factor(Insecticide) * Deposit +
  I(Deposit^2) + I(Deposit^3)
```

	Df	Deviance	AIC
- factor(Insecticide):Deposit	2	14.406	88.604
- I(Deposit^3)	1	14.358	90.556
- I(Deposit^2)	1	14.905	91.102
<none>		14.301	92.498

Step: AIC=88.6

```
cbind(Killed, Number - Killed) ~ factor(Insecticide) + Deposit +
  I(Deposit^2) + I(Deposit^3)
```

	Df	Deviance	AIC
- I(Deposit^3)	1	14.499	86.696

```

- I(Deposit^2)          1   15.160  87.357
<none>                   14.406  88.604
- Deposit                1   18.576  90.774
- factor(Insecticide)    2  207.878 278.075

```

Step: AIC=86.7

```

cbind(Killed, Number - Killed) ~ factor(Insecticide) + Deposit +
  I(Deposit^2)

```

	Df	Deviance	AIC
<none>		14.499	86.696
- I(Deposit^2)	1	48.026	118.223
- Deposit	1	80.710	150.907
- factor(Insecticide)	2	208.131	276.328

```

Call: glm(formula = cbind(Killed, Number - Killed) ~ factor(Insecticide) +
  Deposit + I(Deposit^2), family = binomial, data = deposit)

```

Coefficients:

(Intercept)	factor(Insecticide)B	factor(Insecticide)C
-6.3808	0.3061	2.9264
	Deposit	I(Deposit^2)
	2.1511	-0.1532

Degrees of Freedom: 17 Total (i.e. Null); 13 Residual

Null Deviance: 413.6

Residual Deviance: 14.5 AIC: 86.7

```

> step(min.modele,scope=list(lower=min.modele, upper=max.modele),
  direction="both")

```

Start: AIC=477.84

```

cbind(Killed, Number - Killed) ~ 1

```

	Df	Deviance	AIC
+ Deposit	1	231.56	297.76
+ I(Deposit^2)	1	258.25	324.44
+ factor(Insecticide)	2	259.41	327.61
+ I(Deposit^3)	1	285.10	351.30
<none>		413.64	477.84

Step: AIC=297.76


```
cbind(Killed, Number - Killed) ~ Deposit
```

	Df	Deviance	AIC
+ factor(Insecticide)	2	48.03	118.22
+ I(Deposit^3)	1	207.88	276.08
+ I(Deposit^2)	1	208.13	276.33
<none>		231.56	297.76
- Deposit	1	413.64	477.84

Step: AIC=118.22

```
cbind(Killed, Number - Killed) ~ Deposit + factor(Insecticide)
```

	Df	Deviance	AIC
+ I(Deposit^2)	1	14.499	86.70
+ I(Deposit^3)	1	15.160	87.36
+ factor(Insecticide):Deposit	2	42.773	116.97
<none>		48.026	118.22
- factor(Insecticide)	2	231.559	297.76
- Deposit	1	259.414	327.61

Step: AIC=86.7

```
cbind(Killed, Number - Killed) ~ Deposit + factor(Insecticide) +
  I(Deposit^2)
```

	Df	Deviance	AIC
<none>		14.499	86.696
+ I(Deposit^3)	1	14.406	88.604
+ factor(Insecticide):Deposit	2	14.358	90.556
- I(Deposit^2)	1	48.026	118.223
- Deposit	1	80.710	150.907
- factor(Insecticide)	2	208.131	276.328

```
Call: glm(formula = cbind(Killed, Number - Killed) ~ Deposit +
  factor(Insecticide) + I(Deposit^2), family = binomial, data = deposit)
```

Coefficients:

(Intercept)	Deposit	factor(Insecticide)B
-6.3808	2.1511	0.3061
factor(Insecticide)C	I(Deposit^2)	
2.9264	-0.1532	

Degrees of Freedom: 17 Total (i.e. Null); 13 Residual
Null Deviance: 413.6
Residual Deviance: 14.5 AIC: 86.7

Selon les trois procédures, le modèle retenu est celui qui inclut les deux variables explicatives (type et dose de l'insecticide) et le terme quadratique de la dose.

4 Validation et diagnostique de modèles

4.1 Résidus

Comme pour la régression linéaire, les résidus ont un rôle important dans la validation des modèles. Pour la régression logistique, on distingue deux types de résidus: résidus déviance et résidus Pearson.

Les résidus déviance sont définis par $r_{D,i} = \text{signe}(y_i - \mu_i)\sqrt{d_i}$, où

$$\text{signe}(u) = \begin{cases} 1 & \text{si } u > 0, \\ -1 & \text{si } u < 0, \\ 0 & \text{si } u = 0 \end{cases}$$

et

$$d_i = 2 \times \left\{ y_i \log \left(\frac{y_i}{\hat{\mu}_i} \right) + (m_i - y_i) \log \left(\frac{m_i - y_i}{m_i - \hat{\mu}_i} \right) \right\}.$$

D'autre part, les résidus Pearson sont définis par

$$r_{P,i} = \frac{y_i - m_i \hat{\pi}_i}{\sqrt{m_i \hat{\pi}_i (1 - \hat{\pi}_i)}} = \frac{y_i - \hat{\mu}_i}{\sqrt{\frac{\hat{\mu}_i (m_i - \hat{\mu}_i)}{m_i}}}.$$

4.2 Tests d'adéquation globale

Deux statistiques sont disponibles pour tester l'adéquation globale d'un modèle: la première est la déviance $D = \sum_{i=1}^n r_{D,i}^2$ et la deuxième est la statistique Khi-deux de Pearson

$$\chi_P^2 = \sum_{i=1}^n r_{P,i}^2 = \sum_{i=1}^n \frac{m_i (y_i - \hat{\mu}_i)^2}{\hat{\mu}_i (m_i - \hat{\mu}_i)}.$$

Lorsque tous les $m_i \rightarrow \infty$, les statistiques D et χ^2_P convergent en loi vers une distribution de Khi-deux à $n - (p + 1)$ degrés de liberté si le modèle est bien spécifié. Il est primordial de noter que ces tests ne sont valides que pour des données groupées avec des m_i élevés puisqu'ils reposent sur un résultat asymptotique.

Revenons au jeu de données `deposit` de la library `GLMsData` du logiciel R. À la section précédente, on a retenu le modèle avec les deux variables explicatives (type et dose de l'insecticide) et le terme quadratique de la dose. Testons l'adéquation globale de ce modèle.

```
> if (!require("GLMsData")) install.packages("GLMsData")
Le chargement a ncessit le package : GLMsData
> library("GLMsData")#### Installer la librairie GLMsData
>
> if (!require("statmod")) install.packages("statmod")
Le chargement a ncessit le package : statmod
> library("statmod")#### Installer la librairie statmod
> ## La librairie statmod permet de calculer les residus Pearson
>
> data(deposit)#### Rendre le jeu de donnees deposit disponible
> n <- dim(deposit)[1]
> p <- dim(deposit)[2]
> df <- n-(p+1)
>
> modele <- glm(cbind(Killed,Number-Killed)~factor(Insecticide)+Deposit
+               +I(Deposit^2),data=deposit,family=binomial)
>
> #####
> ## Calcul des residus deviance et Pearson ##
> #####
>
> residus.deviance <- resid(modele)
> residus.Pearson <- qresid(modele)
>
> #####
> ## Calcul des statistiques d'adequation ##
> #####
>
> Deviance <- deviance(modele)
> ## doit donner la meme chose que sum(residus.deviance^2)
```

```

> Pearson <- sum(residus.Pearson^2)
>
> #####
> ## Calcul des seuils observes ##
> #####
>
> pvaleur.Deviance <- 1 - pchisq(Deviance,df=df) ; pvaleur.Deviance
[1] 0.3396796
> pvaleur.Pearson <- 1 - pchisq(Pearson,df=df) ; pvaleur.Pearson
[1] 0.4661418

```

On obtient des seuils observés de 0.339 et 0.466. Le modèle proposé est bien adéquat aux données.

4.3 Graphiques

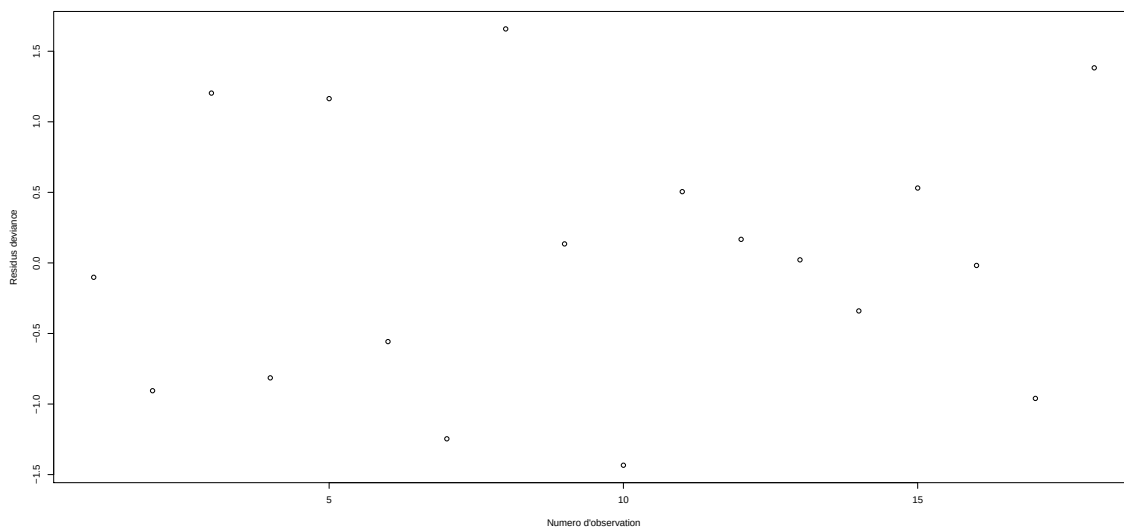
Les graphiques de des résidus en fonction des numéros des observations ne doivent montrer qucune tendance.

```

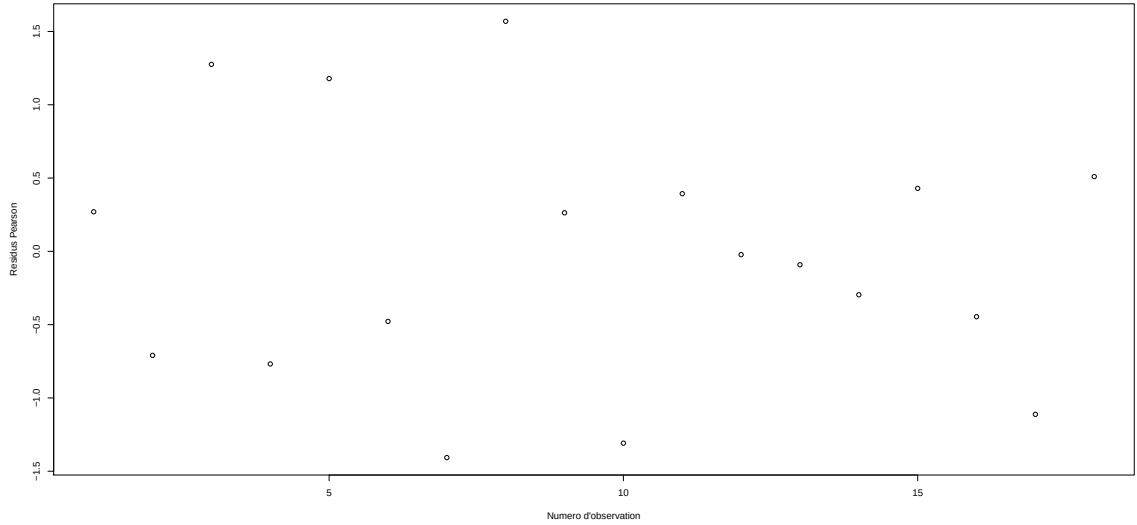
plot(residus.deviance,type="p",xlab="Numero d'observation",ylab="Residus deviance")
plot(residus.Pearson,type="p",xlab="Numero d'observation",ylab="Residus Pearson")

```

On obtient



et



Si ces graphiques ont tendance horizontales mais plusieurs des valeurs sont supérieures à 3 en valeur absolue, cela peut suggérer la présence d'une extra-variabilité. En effet, si la loi binomiale est bien spécifiée, alors on a $E[Y_i] = m_i\pi_i$ et $Var[Y_i] = m_i\pi_i(1 - \pi_i)$. Il arrive dans certaines situations où on a $E[Y_i] = m_i\pi_i$ mais $Var[Y_i] > m_i\pi_i(1 - \pi_i)$. On dit qu'on est en présence d'une extra-variabilité (*overdispersion* en anglais). Dans cette situation, on pose $Var[Y_i] = \phi m_i\pi_i(1 - \pi_i)$ et on estime ϕ par $\chi_P^2/(n - (p + 1))$

```
> Pearson/(n-(p+1))
[1] 0.981905
```

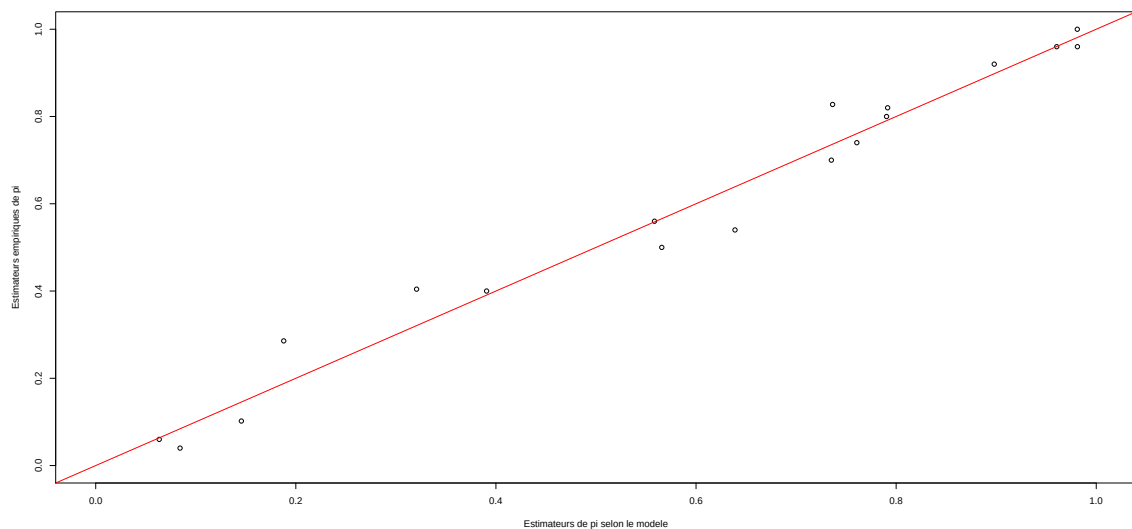
Dans notre cas, la valeur de $\chi_P^2/(n - (p + 1))$ est très proche de 1. On ne détecte aucune extra-variabilité. Une solution à apporter en cas d'une éventuelle extra-variabilité est de spécifier l'option `family=quasibinomial()` au lieu de `family=binomial()`. Cette solution considère une distribution pour Y_i telle que $E[Y_i] = m_i\pi_i$ et $Var[Y_i] = \phi m_i\pi_i(1 - \pi_i)$, avec $\phi > 1$.

Pour vérifier la linéarité $\eta_i = \beta_0 + \beta_1 X_{i1} + \dots + \beta_p X_{ip}$, on trace le nuage de points $\{(\hat{\pi}_i, \tilde{\pi}_i), i = 1 \dots, n\}$. Ici, $\hat{\pi}_i = g^{-1}(\hat{\eta}_i)$ est l'estimateur de π_i selon le modèle considéré et $\tilde{\pi}_i = y_i/m_i$ est l'estimateur empirique de π_i . Si la relation $\eta_i = \beta_0 + \beta_1 X_{i1} + \dots + \beta_p X_{ip}$ est bien spécifiée, les estimateurs $\hat{\pi}_i$

et $\tilde{\pi}_i$ doivent être proches et le nuage de points doit être autour de la droite $x = y$.

```
> pi.chapeau <- fitted(modele)
> pi.tilde <- deposit$Killed/deposit$Number
>
> plot(pi.chapeau,pi.tilde,type="p",xlim=c(0,1),ylim=c(0,1),
+      xlab="Estimateurs de pi selon le modele",ylab="Estimateurs empiriques de pi")
> abline(0,1,col="red")
```

On obtient



4.4 Observations aberrantes & valeurs influentes

Le principe des observations aberrantes et valeurs influentes est identique pour les régressions logistique et linéaire. Les définitions des leviers des observations, des résidus standardisés et studentisés sont différentes dans le cas d'une régression logistique mais ces quantités ont la même interprétation et utilité que dans le cadre d'une régression linéaire. C'est la même chose pour la distance de Cook, les DFBETAS, DFFITS et rapport de ratios.

4.5 Spécificité, sensibilité et courbe ROC

Toutes les procédures de diagnostique et validation de modèle citées ci-haut ne sont valides que pour les données groupées. Dans le cas de données individuelles, c'est plus difficile d'évaluer l'adéquation des modèles. Cependant, il est possible de mesurer la capacité prédictive du modèle.

Considérons un modèle avec des données individuelles où la variable réponse est $Y_i \in \{0, 1\}$, avec $P(Y_i = 1) = \pi_i = g^{-1}(\nu_i)$, $\nu_i = \beta_0 + \beta_1 X_{i1} + \dots + \beta_p X_{ip}$ et $P(Y_i = 0) = 1 - P(Y_i = 1)$. On ajuste le modèle, on obtient $\hat{\beta}_0, \hat{\beta}_1, \dots, \hat{\beta}_p$ et on calcule les probabilités prédites $\hat{\pi}_i = g^{-1}(\hat{\eta}_i)$ avec $\hat{\eta}_i = \hat{\beta}_0 + \hat{\beta}_1 X_{i1} + \dots + \hat{\beta}_p X_{ip}$.

Fixons maintenant un réel $u \in [0, 1]$. Dans un contexte de classification, on pose $\hat{y}_i = 1$ si $\hat{\pi}_i > u$ et $\hat{y}_i = 0$ sinon. La valeur la plus utilisée est $u = 1/2$. On mesure la capacité prédictive du modèle par la spécificité et la sensibilité, définies respectivement par $Spec = P(\hat{y} = 0 | y = 0)$ et $Sens = P(\hat{y} = 1 | y = 1)$. La spécificité et la sensibilité réfèrent aux taux de vrais négatifs et vrais positifs, respectivement. Plus ces valeurs sont proches de 1, plus grande est la capacité prédictive du modèle.

Pour un u fixé, on calcule $\hat{y}_1, \dots, \hat{y}_n$ et on estime la spécificité et la sensibilité du modèle par

$$\begin{aligned}\widehat{Spec}(u) &= \frac{\sum_{i=1}^n I_{\{\hat{y}_i=0\}}}{\sum_{i=1}^n I_{\{(\hat{y}_i=0) \& (y_i=0)\}}} = \frac{\sum_{i=1}^n I_{\{\hat{\pi}_i < u\}}}{\sum_{i=1}^n I_{\{(\hat{\pi}_i < u) \& (y_i=0)\}}} \\ \widehat{Sens}(u) &= \frac{\sum_{i=1}^n I_{\{\hat{y}_i=1\}}}{\sum_{i=1}^n I_{\{(\hat{y}_i=1) \& (y_i=1)\}}} = \frac{\sum_{i=1}^n I_{\{\hat{\pi}_i > u\}}}{\sum_{i=1}^n I_{\{(\hat{\pi}_i > u) \& (y_i=1)\}}}\end{aligned}$$

Faisons maintenant varier u à l'intérieur d'une grille de B points dans $[0, 1]$: $\{u_1 = 0, u_1, u + 2, \dots, u_B = 1\}$, avec B suffisamment grand.

Pour $b = 0, 1, \dots, B$, on calcule $\widehat{Spec}(u_b)$ et $\widehat{Sens}(u_b)$. On trace le nuage de points $\{(1 - \widehat{Spec}(u_b), \widehat{Sens}(u_b)), b = 0, \dots, B\}$. En reliant ces points, on obtient une courbe qu'on appelle la courbe ROC. On calcule l'aire sous cette courbe. Plus cette aire est grande, plus la capacité prédictive du modèle est grande.

$$\text{Aire sous la courbe ROC} \in \begin{cases} [0.9, 1], & \text{le modèle est excellent} \\ [0.8, 0.9], & \text{le modèle est bon} \\ [0.7, 0.8], & \text{le modèle est moyen} \\ [0.6, 0.7], & \text{le modèle est mauvais} \\ [0.5, 0.6], & \text{le modèle est très faible.} \end{cases}$$

Considérons maintenant le jeu de données *quilpie* de la library *GLMsData* du logiciel R. Ce jeu de données contient des observations de 1921 à 1988 de la quantité de pluie mesurée durant le mois de juillet à Quilpie en Australie. On considère une seule variable explicative *SOI* qui mesure la pression atmosphérique. La variable réponse ne peut prendre que 2 valeurs: 1 si la quantité de pluie mesurée est supérieure à 10 millimètres et 0 sinon.

```
> if (!require("GLMsData")) install.packages("GLMsData")
Le chargement a ncessit le package : GLMsData
> library("GLMsData")#### Installer la librairie GLMsData
>
> data(quilpie)
>
> modele <- glm(y~SOI,family=binomial,data=quilpie)
>
> if (!require("clinfun")) install.packages("clinfun")
Le chargement a ncessit le package : clinfun
> library("clinfun")#### Installer la librairie clinfun
>
> courbe.roc <- roc.curve(marker=as.vector(fitted(modele)),
+                         status=quilpie$y,method=c("empirical"))
> print(courbe.roc)
ROC curve with AUC = 0.7835498 and s.e. = 0.05651002
```

On obtient une aire sous la courbe de 0.78 avec une erreur-type de 0.056. On conclut que le modèle est bon. On peut aussi tracer la courbe ROC:

```
> plot(courbe.roc)
```

Et on obtient

